Practical 8: MiniBank Robust Exception Handling Architecture

Problem Definition:

Create a checked exception hierarchy with BankException base class, InsufficientFundsException (carrying shortfall), AccountNotFoundException, and InvalidAmountException; enforce throws declarations in deposit(), withdraw(), and transfer(); and manage sessions with try-with-resources.

Code:

```
class PartB {
    public static class BankException extends Exception {
        public BankException(String message) {
            super(message);
        }
    }

    public static class InsufficientFundsException extends BankException {
        private final long shortfall;

        public InsufficientFundsException(String accNo, long balance, long requested) {
            super("Account [" + accNo + "] has insufficient funds. Balance: Rs. " + balance + ",
Requested: Rs. " + requested);
            this.shortfall = requested - balance;
        }

        public long getShortfall() {
            return shortfall;
        }
    }

    public static class AccountNotFoundException extends BankException {
        public AccountNotFoundException(String accNo) {
            super("Account [" + accNo + "] was not found in the bank directory.");
        }
    }

    public static class InvalidAmountException extends BankException {
        public InvalidAmountException(String message) {
            super(message);
        }
    }

    public static class BankSession implements AutoCloseable {
        private final String sessionId;

        public BankSession(String sessionId) {
            this.sessionId = sessionId;
            System.out.println("  [SESSION] Banking Transaction Session '" + sessionId + "'
opened.");
        }

        @Override
        public void close() {
            System.out.println("  [SESSION] Banking Transaction Session '" + sessionId + "'
safely closed & committed.");
        }
```

```
    }

    public static class Account {
        private final String accountNumber;
        private final String holderName;
        private long balance;

        public Account(String accountNumber, String holderName, long balance) {
            this.accountNumber = accountNumber;
            this.holderName = holderName;
            this.balance = balance;
        }

        public String getAccountNumber() { return accountNumber; }
        public String getHolderName() { return holderName; }
        public long getBalance() { return balance; }

        public void deposit(long amount) throws InvalidAmountException {
            if (amount <= 0) {
                throw new InvalidAmountException("Deposit amount must be strictly greater than
0. Received: Rs. " + amount);
            }
            balance += amount;
            System.out.println("  [DEPOSIT SUCCESS] Deposited Rs. " + amount + " to " +
accountNumber + ". New Balance: Rs. " + balance);
        }

        public void withdraw(long amount) throws InsufficientFundsException,
InvalidAmountException {
            if (amount <= 0) {
                throw new InvalidAmountException("Withdrawal amount must be strictly greater
than 0. Received: Rs. " + amount);
            }
            if (amount > balance) {
                throw new InsufficientFundsException(accountNumber, balance, amount);
            }
            balance -= amount;
            System.out.println("  [WITHDRAW SUCCESS] Withdrawn Rs. " + amount + " from " +
accountNumber + ". Remaining Balance: Rs. " + balance);
        }

        public void transfer(Account recipient, long amount) throws BankException {
            if (recipient == null) {
                throw new AccountNotFoundException("TARGET-ACCOUNT-NULL");
            }
            System.out.println("  [TRANSFER ATTEMPT] Initiating transfer of Rs. " + amount + "
from " + accountNumber + " to " + recipient.accountNumber);
            this.withdraw(amount);
            recipient.deposit(amount);
            System.out.println("  [TRANSFER COMPLETED] Successfully transferred Rs. " + amount +
" to " + recipient.accountNumber);
        }

        @Override
        public String toString() {
            return "[" + accountNumber + " | " + holderName + " | Rs. " + balance + "]";
        }
    }
```

```
    public static void main(String[] args) {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 8 - Part B: MiniBank Robust Exception Handling
Architecture");
        System.out.println();
        Account acc1 = new Account("AC1001", "Rudra Modi", 5000);
        Account acc2 = new Account("AC1002", "Dhyey Patel", 2000);

        System.out.println("Initial State: " + acc1 + " and " + acc2);

        System.out.println("\nTest 1: Invalid Amount Exception on Negative Deposit:");
        try {
            acc1.deposit(-1000);
        } catch (InvalidAmountException e) {
            System.out.println("  [CAUGHT EXCEPTION] " + e.getMessage());
        }

        System.out.println("\nTest 2: Insufficient Funds Exception on Large Withdrawal:");
        try {
            acc1.withdraw(8000);
        } catch (InsufficientFundsException e) {
            System.out.println("  [CAUGHT EXCEPTION] " + e.getMessage());
            System.out.println("  -> Deficit / Shortfall Amount: Rs. " + e.getShortfall());
        } catch (InvalidAmountException e) {
            System.out.println("  [CAUGHT EXCEPTION] " + e.getMessage());
        }

        System.out.println("\nTest 3: Transaction Session with Try-With-Resources & Successful
Transfer:");
        try (BankSession session = new BankSession("TXN-SES-991")) {
            acc1.transfer(acc2, 3000);
        } catch (BankException e) {
            System.out.println("  [TRANSFER FAILED] " + e.getMessage());
        } finally {
            System.out.println("  [AUDIT] Final Balances -> " + acc1 + " | " + acc2);
        }
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-8\PartB.java
RUDRA MODI || 25DCS058

Practical 8 - Part B: MiniBank Robust Exception Handling Architecture

Initial State: [AC1001 | Rudra Modi | Rs. 5000] and [AC1002 | Dhyey Patel | Rs. 2000]

Test 1: Invalid Amount Exception on Negative Deposit:
  [CAUGHT EXCEPTION] Deposit amount must be strictly greater than 0. Received: Rs. -1000

Test 2: Insufficient Funds Exception on Large Withdrawal:
  [CAUGHT EXCEPTION] Account [AC1001] has insufficient funds. Balance: Rs. 5000, Requested: Rs. 8000
  -> Deficit / Shortfall Amount: Rs. 3000

Test 3: Transaction Session with Try-With-Resources & Successful Transfer:
  [SESSION] Banking Transaction Session 'TXN-SES-991' opened.
  [TRANSFER ATTEMPT] Initiating transfer of Rs. 3000 from AC1001 to AC1002
  [WITHDRAW SUCCESS] Withdrawn Rs. 3000 from AC1001. Remaining Balance: Rs. 2000
  [DEPOSIT SUCCESS] Deposited Rs. 3000 to AC1002. New Balance: Rs. 5000
  [TRANSFER COMPLETED] Successfully transferred Rs. 3000 to AC1002
  [SESSION] Banking Transaction Session 'TXN-SES-991' safely closed & committed.
  [AUDIT] Final Balances -> [AC1001 | Rudra Modi | Rs. 2000] | [AC1002 | Dhyey Patel | Rs. 5000]
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: Why are business errors like InsufficientFundsException designed as checked exceptions?

Answer: Checked exceptions require callers to explicitly anticipate and handle predictable banking failures (such as overdraft or invalid account lookups) using try-catch blocks, ensuring robust transaction safety.

Q2: How does try-with-resources ensure BankSession closure?

Answer: BankSession implements the AutoCloseable interface, allowing it to be managed within a try-with-resources statement where close() is invoked deterministically upon exit, guaranteeing transaction auditing and cleanup.